FPT UNIVERSITY
FACULTY OF INFORMATION TECHNOLOGY

GROUP FINAL PROJECT
FAP201

THIẾT KẾ VÀ MÔ PHỎNG IP TIMER 64-BIT GIAO TIẾP AMBA APB

Lecturer: Phạm Thế Vinh

Class: IC2007

Topic: IP Timer (AMBA APB)

Students:

Phùng Văn Tân - SE203222

Hoàng Trung Hiếu - SE203208

Hoàng Quốc Trí - SE203836

Lê Dương Hoàng Khang - SE204508

Trần Định Quốc - CE202008

Ho Chi Minh City, Ngày 1 Tháng 11 Năm 2026

LỜI CẢM ƠN

Đầu tiên, chúng em xin gửi lời cảm ơn chân thành và sâu sắc nhất đến Thầy Phạm Thế Vinh – giảng viên Trường Đại học FPT, người đã tận tình hướng dẫn, cung cấp tài liệu đặc tả và định hướng phương pháp thiết kế, kiểm thử vi mạch cho chúng em trong suốt học phần FAP201.

Trong quá trình học tập tại Trường Đại học FPT, chúng em đã được truyền đạt những kiến thức nền tảng quý báu về lĩnh vực Thiết kế Vi mạch Bán dẫn: từ tư duy mô tả phần cứng bằng Verilog, chuẩn giao tiếp bus AMBA cho tới quy trình kiểm thử tự động. Những kiến thức và kỹ năng đó không chỉ giúp nhóm hoàn thành đồ án "Thiết kế và Mô phỏng IP Timer 64-bit giao tiếp AMBA APB", mà còn là hành trang để mỗi thành viên tự tin theo đuổi con đường Kỹ sư Thiết kế và Kiểm thử Vi mạch (RTL Design / Verification Engineer) trong tương lai.

Chúng em cũng xin cảm ơn nhóm Trương Vũ (lớp IC2007) đã chia sẻ cách tổ chức mã nguồn và trình bày báo cáo đồ án UART-FIFO, giúp nhóm có một hình mẫu rõ ràng khi xây dựng repository và báo cáo này. Cảm ơn từng thành viên trong nhóm đã phối hợp chặt chẽ, hỗ trợ nhau trong suốt quá trình thiết kế, tích hợp và gỡ lỗi.

Cuối cùng, chúng em xin gửi lời cảm ơn đến gia đình, người thân và bạn bè – những người đã luôn là chỗ dựa tinh thần vững chắc giúp chúng em vượt qua những thử thách để hoàn thành tốt đồ án này.

Trân trọng,

Thành phố Hồ Chí Minh, Ngày 1 Tháng 11 Năm 2026

Thay mặt nhóm thực hiện

Hoàng Quốc Trí
Khoa Vi Mạch, Bán Dẫn - Lớp IC2007

MỤC LỤC

DANH MỤC HÌNH.....5
DANH MỤC BẢNG.....6
DANH MỤC TỪ VIẾT TẮT.....7
TÓM TẮT BÀI BÁO CÁO.....8
Chương 1. GIỚI THIỆU TỔNG QUAN ĐỀ TÀI.....9
1.1. Bối cảnh và nhu cầu thực tế.....9
1.2. Mục tiêu và phương pháp thực hiện đề tài.....10
1.2.1. Mục tiêu của đề tài.....10
1.2.2. Phương pháp thực hiện.....10
1.3. Bảng thông số kỹ thuật của hệ thống.....11
1.4. Phân công công việc trong nhóm.....11
Chương 2. CƠ SỞ LÝ THUYẾT.....13
2.1. Giao thức bus AMBA APB.....13
2.1.1. Tổng quan và các tín hiệu của APB.....13
2.1.2. Máy trạng thái truyền dữ liệu APB.....13
2.1.3. Giao dịch ghi/đọc có chèn chu kỳ chờ.....14
2.2. Bộ định thời (Timer) và bộ chia tần (Prescaler).....14
2.2.1. Nguyên lý bộ đếm 64-bit.....14
2.2.2. Chia tần bằng xung cho phép đếm (Clock Enable).....14
2.3. Cơ chế ngắt và thanh ghi ghi-1-để-xóa (RW1C).....15
2.4. Chế độ dừng gỡ lỗi (Debug Halt).....15
2.5. Phương pháp kiểm thử tự động (Self-checking Testbench).....15
Chương 3. THIẾT KẾ PHẦN CỨNG.....17
3.1. Sơ đồ khối tổng thể của IP Timer.....17
3.2. Module timer_top.....18
3.3. Bản đồ thanh ghi (Register Map).....18
3.4. Thiết kế chi tiết module giao tiếp APB (apb_slave).....20
3.4.1. Module apb_slave và nguyên lý hoạt động.....20
3.4.2. Cơ chế chèn chu kỳ chờ và phản hồi lỗi.....21
3.5. Thiết kế chi tiết module thanh ghi (timer_register).....21
3.5.1. Module timer_register và nguyên lý hoạt động.....21
3.5.2. Ghi theo byte strobe.....23
3.5.3. Kiểm tra lỗi và bảo vệ cấu hình TCR.....23
3.5.4. Thanh ghi trạng thái ngắt TISR (RW1C).....23
3.6. Thiết kế chi tiết module chia tần (counter_divider).....24
3.6.1. Module counter_divider và nguyên lý hoạt động.....24
3.6.2. Sơ đồ logic của bộ chia tần.....24
3.7. Thiết kế chi tiết module lõi đếm (counter_core).....25
3.7.1. Module counter_core và nguyên lý hoạt động.....25
3.7.2. Sơ đồ logic và thứ tự ưu tiên cập nhật bộ đếm.....26

3.8. Tích hợp hệ thống và kết quả tổng hợp logic.....27
Chương 4. KẾT QUẢ KIỂM THỬ VÀ MÔ PHỎNG HỆ THỐNG.....29
4.1. Môi trường kiểm thử tự động (Self-checking Testbench).....29
4.2. Kết quả mô phỏng trên phần mềm ModelSim.....30
4.2.1. Kịch bản truy cập APB và thanh ghi (TEST 1 – TEST 4).....31
4.2.2. Kịch bản ngắt so sánh và dừng gỡ lỗi (TEST 5 – TEST 6).....32
4.2.3. Kịch bản bộ chia tần và reset bất đồng bộ (TEST 7 – TEST 8).....32
Chương 5. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN CỦA ĐỀ TÀI.....33
5.1. Kết luận.....33
5.2. Những khó khăn trong quá trình thiết kế.....33
5.3. Hướng phát triển trong tương lai.....34
TÀI LIỆU THAM KHẢO.....35

DANH MỤC HÌNH

Hình 1.1: Vị trí của IP Timer trong hệ thống SoC giao tiếp qua bus AMBA APB.....9
Hình 2.1: Sơ đồ máy trạng thái truyền dữ liệu của giao thức APB.....14
Hình 2.2: Giản đồ thời gian giao dịch ghi và đọc APB có chèn 1 chu kỳ chờ.....14
Hình 2.3: Sơ đồ môi trường kiểm thử (Testbench) của IP Timer.....16
Hình 3.1: Sơ đồ khối kiến trúc tổng thể của IP Timer (timer_top).....17
Hình 3.2: Khối timer_top.....18
Hình 3.3: Cấu trúc trường bit của các thanh ghi TCR, TIER, TISR và THCSR.....19
Hình 3.4: Khối apb_slave.....20
Hình 3.5: Đoạn mã tạo chu kỳ chờ và các tín hiệu phản hồi của apb_slave.....21
Hình 3.6: Khối timer_register.....22
Hình 3.7: Đoạn mã kiểm tra lỗi TCR và cập nhật TISR.....23
Hình 3.8: Khối counter_divider.....24
Hình 3.9: Sơ đồ logic của khối counter_divider.....25
Hình 3.10: Khối counter_core.....26
Hình 3.11: Sơ đồ logic của khối counter_core.....27
Hình 3.12: Đoạn mã cập nhật bộ đếm 64-bit của counter_core.....27
Hình 4.1: Kết quả kiểm tra tự động của khối timer_top trên ModelSim (checks = 45, errors = 0).....30
Hình 4.2: Mô phỏng Waveform toàn bộ quá trình kiểm thử của timer_top_tb trên ModelSim.....31
Hình 5.1: Ảnh chụp ModelSim khi nạp các module RTL như top-level riêng – tín hiệu hiển thị HiZ/StX.....34

DANH MỤC BẢNG

Bảng 1.1: Thông số kỹ thuật của IP Timer....11
Bảng 1.2: Phân công công việc trong nhóm....12
Bảng 2.1: Các tín hiệu của giao thức AMBA APB và tên tương ứng trong thiết kế....13
Bảng 2.2: So sánh hai phương pháp chia tần....15
Bảng 3.1: Đặc tả khối timer_top....18
Bảng 3.2: Bản đồ thanh ghi (Register Map) của IP Timer....19
Bảng 3.3: Mô tả chi tiết các trường bit điều khiển....20
Bảng 3.4: Đặc tả khối apb_slave....21
Bảng 3.5: Đặc tả khối timer_register....22
Bảng 3.6: Các trường hợp ghi thanh ghi TCR và phản hồi của IP....23
Bảng 3.7: Đặc tả khối counter_divider....24
Bảng 3.8: Quan hệ giữa div_val, ngưỡng đếm và chu kỳ tick (sys_clk = 100 MHz)....25
Bảng 3.9: Đặc tả khối counter_core....26
Bảng 3.10: Kết quả tổng hợp logic (Yosys) của IP Timer....28
Bảng 4.1: Các nhóm kiểm thử của testbench timer_top_tb....29
Bảng 4.2: Đối chiếu các nhóm kiểm thử với dạng sóng ModelSim....31

DANH MỤC TỪ VIẾT TẮT

AHB	Advanced High-performance Bus
AMBA	Advanced Microcontroller Bus Architecture
APB	Advanced Peripheral Bus
ASIC	Application-Specific Integrated Circuit
AXI	Advanced eXtensible Interface
CDC	Clock Domain Crossing
CPU	Central Processing Unit
DUT	Design Under Test
FF	Flip-Flop
FPGA	Field-Programmable Gate Array
FSM	Finite State Machine
HW	Hardware
IC	Integrated Circuit
IP	Intellectual Property
LSB	Least Significant Bit
MSB	Most Significant Bit
RO	Read Only
RTL	Register Transfer Level
RW	Read / Write
RW1C	Read / Write-1-to-Clear
SoC	System on Chip
SW	Software
TB	Testbench
TCMP	Timer Compare Register
TCR	Timer Control Register
TDR	Timer Data Register
THCSR	Timer Halt Control and Status Register
TIER	Timer Interrupt Enable Register
TISR	Timer Interrupt Status Register
W1C	Write-1-to-Clear

TÓM TẮT BÀI BÁO CÁO

Bộ định thời (Timer) là khối ngoại vi nền tảng của mọi vi điều khiển và hệ thống trên chip (SoC), cung cấp nhịp thời gian, ngắt định kỳ và khả năng đo thời gian cho phần mềm. Đồ án trình bày quá trình thiết kế và kiểm thử một lõi IP Timer 64-bit giao tiếp với CPU thông qua bus AMBA APB.

Thiết kế được mô tả bằng Verilog theo cấu trúc module hóa: giao tiếp APB (apb_slave) có chèn chu kỳ chờ và phản hồi lỗi, khối thanh ghi (timer_register) hỗ trợ ghi theo byte strobe và cờ ngắt RW1C, bộ chia tần (counter_divider) theo hệ số 2^div_val với cơ chế bảo vệ cấu hình, và lõi đếm (counter_core) 64-bit với bộ so sánh tạo ngắt và chế độ dừng gỡ lỗi (Debug Halt). Các khối được tích hợp trong module timer_top chạy trên một miền xung nhịp duy nhất.

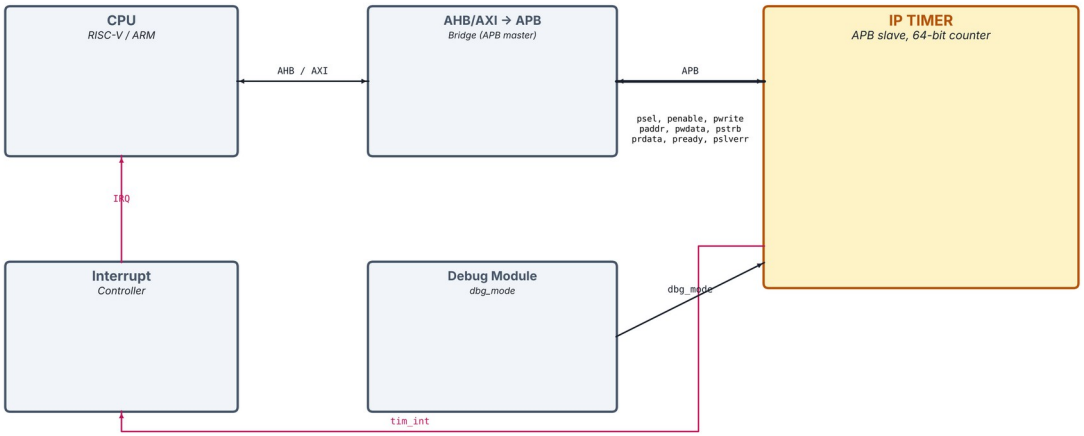
Thiết kế được xác minh bằng môi trường kiểm thử tự kiểm tra (Self-checking Testbench) timer_top_tb và mô phỏng trên phần mềm ModelSim. Kết quả: 8 nhóm kiểm thử với 45 điểm kiểm tra đều đạt (checks = 45, errors = 0); dạng sóng thể hiện đúng giao thức APB, cơ chế ngắt, dừng gỡ lỗi, bộ chia tần và reset bất đồng bộ. Thiết kế tổng hợp thành công với 210 flip-flop, không sinh latch, sẵn sàng để tích hợp vào SoC và triển khai thử nghiệm trên FPGA.

Chương 1. GIỚI THIỆU TỔNG QUAN ĐỀ TÀI

1.1. Bối cảnh và nhu cầu thực tế

Trong mọi vi điều khiển (Microcontroller) và hệ thống trên chip (SoC), bộ định thời (Timer) là một trong những khối ngoại vi không thể thiếu. Timer cung cấp "nhịp thời gian" cho toàn hệ thống: tạo ngắt định kỳ (system tick) cho hệ điều hành thời gian thực, đo khoảng thời gian giữa hai sự kiện, tạo độ trễ chính xác cho phần mềm hay giám sát quá thời gian (timeout) của các giao tiếp ngoại vi. Bộ xử lý trung tâm (CPU) truy cập Timer giống như truy cập bộ nhớ: đọc/ghi các thanh ghi được ánh xạ địa chỉ thông qua bus hệ thống.

Trong kiến trúc SoC dựa trên chuẩn AMBA của ARM, các ngoại vi tốc độ thấp như Timer, UART, GPIO được gắn trên bus **APB (Advanced Peripheral Bus)** – một bus đơn giản, tiêu thụ ít năng lượng, nối với bus hệ thống tốc độ cao (AHB/AXI) thông qua cầu nối (Bridge). Vị trí của IP Timer trong một SoC điển hình được thể hiện ở Hình 1.1.



Hình 1.1: Vị trí của IP Timer trong hệ thống SoC giao tiếp qua bus AMBA APB.

Một Timer 32-bit chạy ở tần số 100 MHz sẽ tràn sau khoảng 2^{32} / 100 MHz ≈ 42,9 giây, buộc phần mềm phải xử lý tràn số liên tục. Với bộ đếm **64-bit**, thời gian tràn kéo dài tới khoảng 5.845 năm, đủ để dùng như một đồng hồ hệ thống suốt vòng đời sản phẩm. Bên cạnh đó, khi lập trình viên dừng CPU để gỡ lỗi (debug), Timer cũng cần có khả năng "đóng băng" để trạng thái hệ thống không bị sai lệch – đây là tính năng **Debug Halt**.

Xuất phát từ nhu cầu đó, đề tài thực hiện thiết kế và kiểm thử một lõi **IP Timer 64-bit giao tiếp AMBA APB** ở mức RTL bằng Verilog, có bộ chia tần lập trình được, ngắt so sánh và chế độ dừng gỡ lỗi, được kiểm chứng bằng testbench tự kiểm tra trên phần mềm ModelSim.

1.2. Mục tiêu và phương pháp thực hiện đề tài

1.2.1. Mục tiêu của đề tài

Đề tài hướng tới việc thiết kế thành công một lõi IP Timer có thể tích hợp vào SoC với các mục tiêu cụ thể như sau:

- **Giao tiếp AMBA APB:** Hỗ trợ các tín hiệu `tim_pstrb` (ghi theo byte), `tim_pready` (chèn 1 chu kỳ chờ) và `tim_pslverr` (báo lỗi), địa chỉ 12-bit, dữ liệu 32-bit.
- **Bộ đếm 64-bit đọc/ghi được:** Bộ đếm lên 64-bit truy cập qua hai thanh ghi 32-bit TDR0/TDR1, cho phép ghi theo từng byte.
- **Bộ chia tần (Prescaler) có bảo vệ cấu hình:** Chia tần số đếm theo 2^div_val (div_val = 0..7), từ chối giá trị 8..15 và mọi thay đổi div_en/div_val khi timer đang chạy.
- **Ngắt so sánh 64-bit:** So sánh bộ đếm với TCMP1:TCMP0, cờ ngắt TISR.int_st xóa bằng cơ chế ghi 1 để xóa (RW1C), có thể che ngắt bằng TIER.int_en.
- **Debug Halt:** Đóng băng bộ chia tần và bộ đếm khi hệ thống ở chế độ gỡ lỗi và phần mềm yêu cầu dừng.
- **Kiểm chứng trên ModelSim:** Xây dựng testbench tự kiểm tra, chạy trên ModelSim với kết quả 0 lỗi.

1.2.2. Phương pháp thực hiện

Đề tài tập trung vào khâu Front-end (RTL Design & Verification), không triển khai trực tiếp lên phần cứng FPGA. Phương pháp thực hiện được tiếp cận theo quy trình thiết kế vi mạch:

- **Thiết kế module hóa:** IP được chia thành các khối chức năng độc lập: giao tiếp APB (apb_slave), khối thanh ghi (timer_register), bộ chia tần (counter_divider) và lõi đếm (counter_core), được tích hợp trong module timer_top. Mỗi thành viên phụ trách một khối.
- **Ngôn ngữ mô tả phần cứng:** Verilog-2001, tách biệt logic tổ hợp (`assign`, `always @(*)`) và logic tuần tự (`always @(posedge sys_clk or negedge sys_rst_n)`).
- **Kiểm thử tự động:** Testbench timer_top_tb đóng vai CPU, phát các giao dịch APB, tự so sánh kết quả với giá trị kỳ vọng và đếm số điểm kiểm tra (checks) cùng số lỗi (errors). Mô hình tham chiếu GOLDEN_MODEL.py đi kèm đề bài được dùng để đối chiếu hành vi của bộ chia tần và bộ đếm.

- **Mô phỏng trên ModelSim:** Thiết kế và testbench được biên dịch, mô phỏng và quan sát dạng sóng trên ModelSim (script run.do, project TONG_HOP_CODE_IP_TIMER.mpf).

1.3. Bảng thông số kỹ thuật của hệ thống

Cấu hình của IP Timer được sử dụng để thiết kế và kiểm thử trong đề tài được thể hiện qua Bảng 1.1.

Thông số	Giá trị	Mô tả chi tiết
Giao tiếp bus	AMBA APB slave	Có PSTRB, PREADY, PSLVERR.
Độ rộng địa chỉ	12 bit	Sử dụng các offset 0x000 – 0x01C; hai bit thấp được bỏ qua.
Độ rộng dữ liệu	32 bit	Ghi theo byte bằng tim_pstrb[3:0].
Chu kỳ chờ	1 chu kỳ	Mỗi giao dịch APB mất 3 chu kỳ (SETUP + 2 chu kỳ ACCESS).
Bộ đếm	64 bit, đếm lên	Đọc/ghi qua TDR0 (bit 31:0) và TDR1 (bit 63:32).
Bộ chia tần	2^div_val	div_val = 0..7 (chia 1 đến chia 128); 8..15 bị từ chối.
Giá trị so sánh	64 bit	TCMP1:TCMP0, giá trị reset 0xFFFF_FFFF_FFFF_FFFF.
Ngắt	1 ngõ ra tim_int	tim_int = int_st & int_en; TISR xóa bằng RW1C.
Debug Halt	halt_ack	halt_ack = dbg_mode & halt_req, đóng băng bộ đếm.
Clock / Reset	sys_clk / sys_rst_n	Một miền clock; reset bất đồng bộ, tích cực mức thấp.
Clock mô phỏng	100 MHz	Chu kỳ 10 ns trong testbench.
Tài nguyên (Yosys)	210 flip-flop	1.209 cell logic chung, không có latch.

Bảng 1.1: Thông số kỹ thuật của IP Timer.

1.4. Phân công công việc trong nhóm

Đề tài được thực hiện bởi nhóm 5 thành viên. Mỗi khối RTL được giao cho một thành viên chịu trách nhiệm thiết kế; thành viên phụ trách tích hợp hệ thống xây dựng timer_top, testbench và thực hiện mô phỏng trên ModelSim (Bảng 1.2).

STT	Họ và tên	MSSV	Nhiệm vụ	Mã nguồn chính
1	Phùng Văn Tân	SE203222	Timer Divider	counter_divider (counter.v)
2	Hoàng Trung Hiếu	SE203208	APB Slave Interface	apb_slave.v
3	Hoàng Quốc Trí	SE203836	System Integration and Verification	timer_top.v timer_top_tb.v
4	Lê Dương Hoàng Khang	SE204508	Timer Core	counter_core, counter (counter.v)
5	Trần Định Quốc	CE202008	Register Bank	timer_register (register.v)

Bảng 1.2: Phân công công việc trong nhóm.

Chương 2. CƠ SỞ LÝ THUYẾT

2.1. Giao thức bus AMBA APB

2.1.1. Tổng quan và các tín hiệu của APB

AMBA (Advanced Microcontroller Bus Architecture) là bộ chuẩn bus trên chip do ARM phát triển, gồm AXI, AHB và APB. Trong đó **APB (Advanced Peripheral Bus)** được tối ưu cho các ngoại vi tốc độ thấp: không có cơ chế đường ống (non-pipelined), mọi tín hiệu đều được lấy mẫu tại sườn lên của PCLK và mỗi giao dịch kéo dài tối thiểu hai chu kỳ. Phiên bản APB4 bổ sung tín hiệu PSTRB (ghi theo byte). IP Timer trong đề tài hiện thực vai trò **APB slave**, các tín hiệu được liệt kê trong Bảng 2.1.

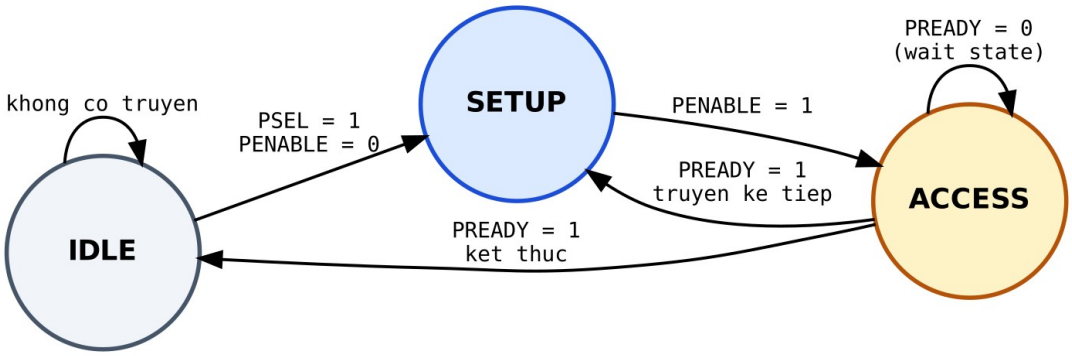
Tín hiệu APB	Tên trong thiết kế	Nguồn	Mô tả
PCLK	sys_clk	Hệ thống	Xung nhịp, mọi tín hiệu lấy mẫu ở sườn lên.
PRESETn	sys_rst_n	Hệ thống	Reset tích cực mức thấp.
PSELx	tim_psel	Master	Chọn slave, bắt đầu giao dịch.
PENABLE	tim_penable	Master	Báo hiệu pha ACCESS.
PWRITE	tim_pwrite	Master	1 = ghi, 0 = đọc.
PADDR	tim_paddr[11:0]	Master	Địa chỉ thanh ghi (offset byte).
PWDATA	tim_pwdata[31:0]	Master	Dữ liệu ghi.
PSTRB	tim_pstrb[3:0]	Master	Byte nào của PWDATA được ghi.
PRDATA	tim_prdata[31:0]	Slave	Dữ liệu đọc, master lấy mẫu khi PREADY = 1.
PREADY	tim_pready	Slave	Slave hoàn tất giao dịch; = 0 để chèn chu kỳ chờ.
PSLVERR	tim_pslverr	Slave	Báo lỗi giao dịch, chỉ có nghĩa ở chu kỳ cuối.

Bảng 2.1: Các tín hiệu của giao thức AMBA APB và tên tương ứng trong thiết kế.

2.1.2. Máy trạng thái truyền dữ liệu APB

Hoạt động của bus APB được mô tả bằng máy trạng thái ba trạng thái (Hình 2.1):

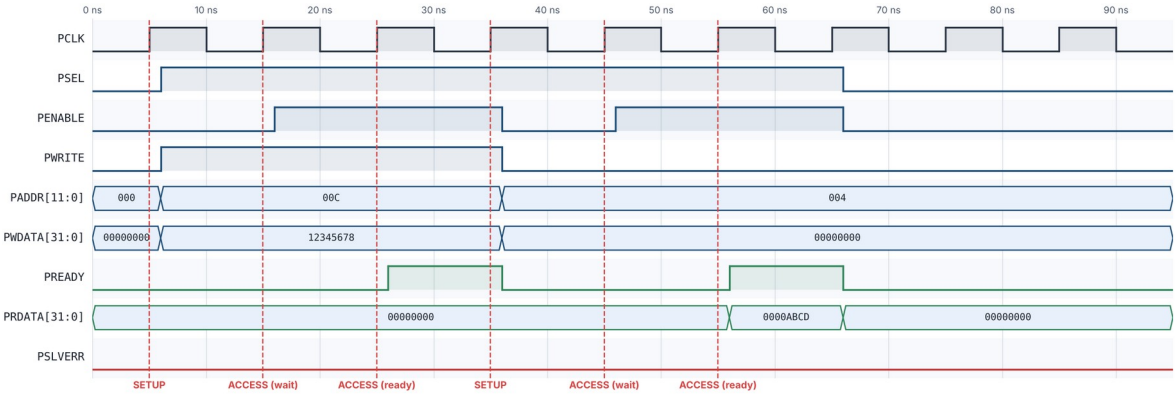
- **IDLE:** Trạng thái mặc định, không có giao dịch (PSEL = 0).
- **SETUP:** Master đặt PSEL = 1, PENABLE = 0 cùng địa chỉ, chiều và dữ liệu. Trạng thái này chỉ kéo dài đúng một chu kỳ.
- **ACCESS:** Master đặt PENABLE = 1 và giữ ổn định mọi tín hiệu. Nếu PREADY = 0, bus ở lại ACCESS (wait state); khi PREADY = 1, giao dịch hoàn tất và bus chuyển về IDLE hoặc sang SETUP nếu có giao dịch kế tiếp.



Hình 2.1: Sơ đồ máy trạng thái truyền dữ liệu của giao thức APB.

2.1.3. Giao dịch ghi/đọc có chèn chu kỳ chờ

Hình 2.2 minh họa một lệnh ghi theo sau là một lệnh đọc khi slave chèn 1 chu kỳ chờ. Ở chu kỳ SETUP, master đưa địa chỉ và dữ liệu; chu kỳ ACCESS đầu tiên slave giữ PREADY = 0; tới chu kỳ ACCESS thứ hai PREADY = 1, dữ liệu ghi được chốt (với lệnh ghi) hoặc PRDATA được master lấy mẫu (với lệnh đọc). Như vậy mỗi giao dịch của IP Timer mất đúng 3 chu kỳ sys_clk. Tín hiệu PSLVERR chỉ được xem xét ở chu kỳ cuối cùng khi PSEL, PENABLE và PREADY đều bằng 1.



Hình 2.2: Giản đồ thời gian giao dịch ghi và đọc APB có chèn 1 chu kỳ chờ.

2.2. Bộ định thời (Timer) và bộ chia tần (Prescaler)

2.2.1. Nguyên lý bộ đếm 64-bit

Lõi của Timer là một thanh ghi đếm lên: mỗi khi có điều kiện cho phép, giá trị tăng thêm 1. Phần mềm đọc giá trị đếm để biết thời gian đã trôi qua, hoặc nạp một giá trị so sánh để được báo ngắt khi bộ đếm chạm tới. Vì bus dữ liệu chỉ rộng 32 bit, bộ đếm 64-bit được ánh xạ thành hai thanh ghi: **TDR0** chứa 32 bit thấp và **TDR1** chứa 32 bit cao.

2.2.2. Chia tần bằng xung cho phép đếm (Clock Enable)

Có hai cách làm bộ đếm chạy chậm hơn sys_clk: tạo ra một xung clock mới có tần số thấp hơn, hoặc giữ nguyên clock và chỉ cho phép đếm ở một số chu kỳ nhất định. Cách thứ nhất sinh ra miền clock mới, gây lệch pha (clock skew), phải xử lý giao tiếp giữa các miền clock (CDC) và làm phức tạp phân tích định thời tĩnh. Vì vậy thiết kế sử dụng cách

thứ hai: khối chia tần chỉ tạo ra **xung div_tick rộng 1 chu kỳ** làm tín hiệu cho phép đếm, toàn bộ IP vẫn chạy chung một miền clock sys_clk (Bảng 2.2).

Tiêu chí	Tạo clock mới	Xung cho phép đếm (div_tick)
Số miền clock	Nhiều miền clock	Một miền clock sys_clk
Đồng bộ dữ liệu (CDC)	Bắt buộc phải xử lý	Không cần
Phân tích định thời	Phức tạp (generated clock)	Đơn giản
Thay đổi hệ số chia	Dễ sinh xung clock lỗi (glitch)	An toàn, chỉ đổi ngưỡng đếm

Bảng 2.2: So sánh hai phương pháp chia tần.

Tần số đếm của bộ đếm 64-bit được xác định theo công thức:

$$f_{count} = f_{sys_clk} / 2^{div_val} \quad \text{(khi div_en = 1)}; \qquad f_{count} = f_{sys_clk} \quad \text{(khi div_en = 0)}$$

Ví dụ với sys_clk = 100 MHz và div_val = 7 (giá trị lớn nhất), bộ đếm tăng sau mỗi 128 chu kỳ (1,28 µs), tương ứng tần số đếm 781,25 kHz.

2.3. Cơ chế ngắt và thanh ghi ghi-1-để-xóa (RW1C)

Khi bộ đếm bằng giá trị so sánh, phần cứng bật **cờ trạng thái ngắt** int_st. Cờ này được chốt (latched): đã bật thì giữ nguyên cho tới khi phần mềm chủ động xóa, kể cả khi bộ đếm đã vượt qua giá trị so sánh. Ngõ ra ngắt gửi tới CPU được tạo bởi phép AND giữa cờ trạng thái và bit cho phép ngắt: `tim_int = int_st & int_en`. Nhờ đó phần mềm có thể che ngắt (int_en = 0) nhưng vẫn thăm dò (polling) được cờ int_st.

Thanh ghi trạng thái ngắt TISR dùng kiểu truy cập **RW1C (Read / Write-1-to-Clear)**: ghi 1 vào bit int_st sẽ xóa cờ, ghi 0 không có tác dụng. Cơ chế này cho phép phần mềm xóa chính xác cờ đã xử lý mà không cần thao tác đọc – sửa – ghi. Khi lệnh xóa của phần mềm và sự kiện bật cờ của phần cứng xảy ra trong cùng một chu kỳ, thiết kế phải quy định rõ bên nào được ưu tiên (mục 3.5.4).

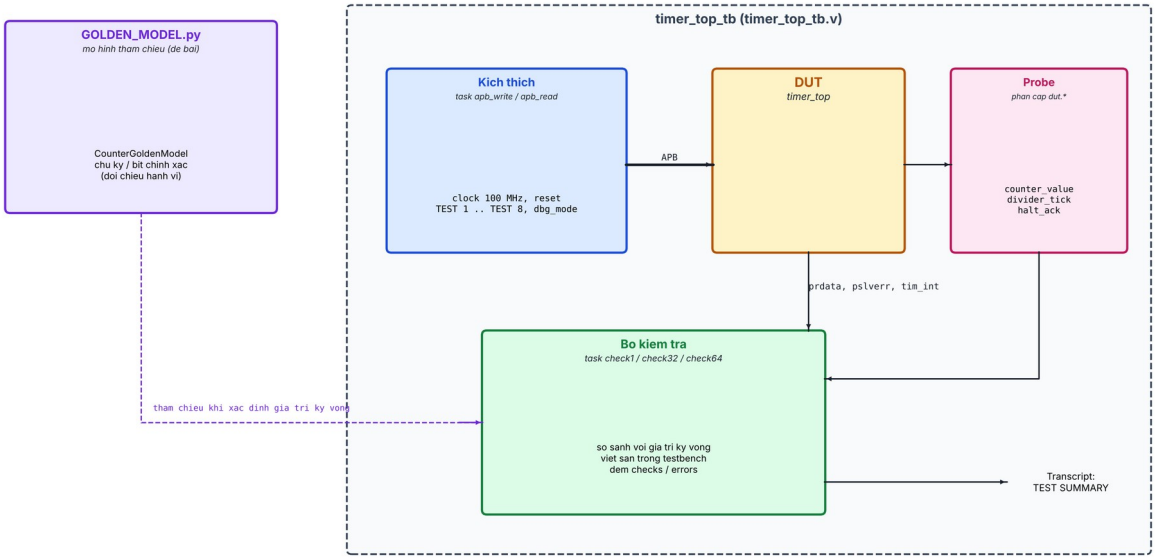
2.4. Chế độ dừng gỡ lỗi (Debug Halt)

Khi lập trình viên dừng CPU tại một điểm ngắt (breakpoint), nếu Timer vẫn tiếp tục đếm thì sau khi chạy tiếp, phần mềm sẽ thấy một khoảng thời gian "nhảy vọt" và có thể phát sinh ngắt không mong muốn. Vì vậy IP Timer hỗ trợ chế độ Debug Halt với cơ chế bắt tay hai tín hiệu: phần mềm đặt bit **halt_req** trong thanh ghi THCSR để cho phép dừng, còn tín hiệu phần cứng **dbg_mode** báo hệ thống đang ở chế độ gỡ lỗi. Chỉ khi cả hai cùng bằng 1, Timer mới dừng và báo lại bằng bit chỉ đọc **halt_ack = dbg_mode & halt_req**. Trong thời gian dừng, cả bộ đếm chia tần lẫn bộ đếm 64-bit đều giữ nguyên giá trị; khi một trong hai điều kiện được gỡ bỏ, Timer đếm tiếp từ đúng giá trị đã dừng.

2.5. Phương pháp kiểm thử tự động (Self-checking Testbench)

Thay vì quan sát dạng sóng bằng mắt một cách thủ công, đề tài xây dựng môi trường kiểm thử tự kiểm tra. Testbench đóng vai trò CPU: phát các giao dịch APB theo

đúng giao thức, đọc lại kết quả và tự so sánh với giá trị kỳ vọng được xác định từ đặc tả và mô hình tham chiếu GOLDEN_MODEL.py (cung cấp kèm đề bài). Mỗi lần so sánh được tính là một điểm kiểm tra; mọi sai khác đều được báo FAIL kèm thời điểm xảy ra và cộng vào bộ đếm lỗi (Hình 2.3).

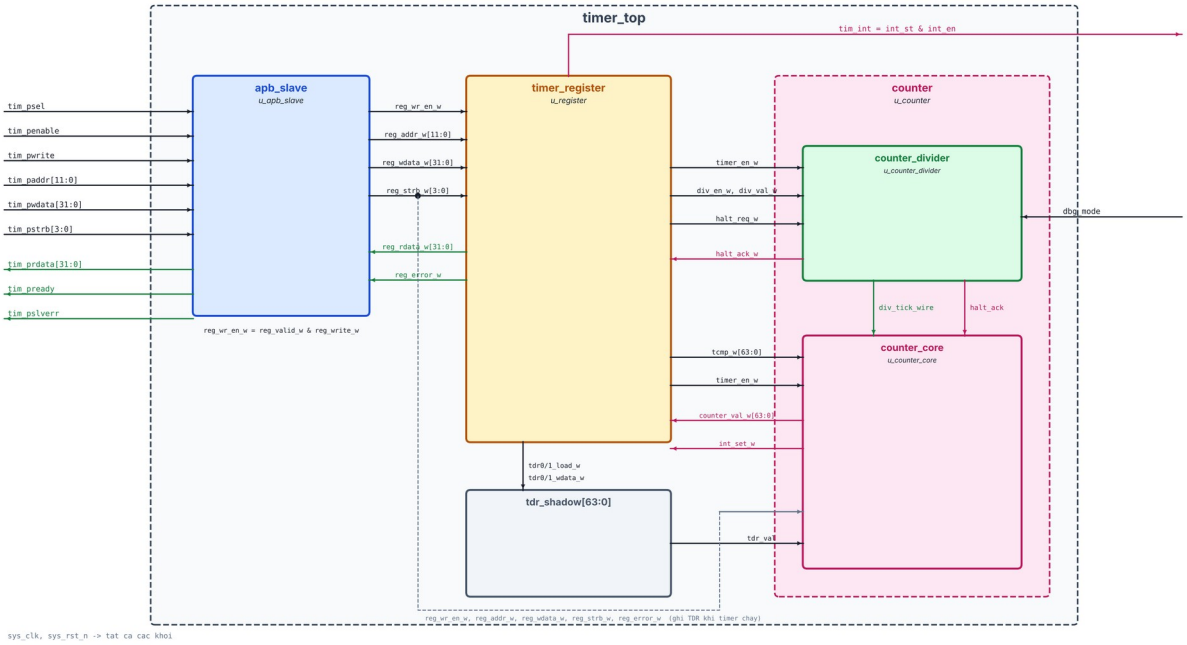


Hình 2.3: Sơ đồ môi trường kiểm thử (Testbench) của IP Timer.

Ưu điểm của phương pháp này là khả năng kiểm tra nhiều trường hợp chỉ trong một lần chạy, có thể chạy lại tự động mỗi khi sửa RTL, và kết quả được tổng hợp thành một dòng TEST SUMMARY rõ ràng trong Transcript của ModelSim.

Chương 3. THIẾT KẾ PHẦN CỨNG

3.1. Sơ đồ khối tổng thể của IP Timer



Hình 3.1: Sơ đồ khối kiến trúc tổng thể của IP Timer (timer_top).

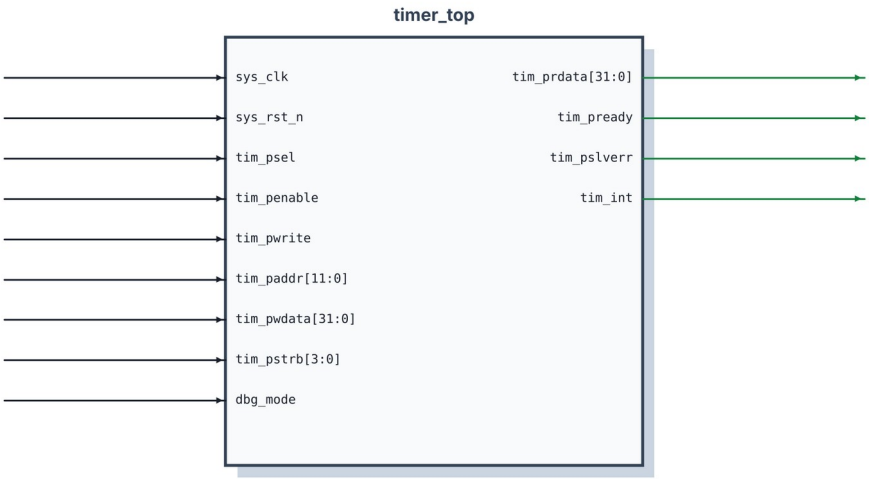
IP Timer được thiết kế theo cấu trúc module hóa (Modular Design) nhằm tăng khả năng tái sử dụng và cho phép từng thành viên phát triển độc lập. Kiến trúc tổng thể gồm các khối chức năng được đóng gói trong module tầng cao nhất timer_top:

- **apb_slave (u_apb_slave):** Giao tiếp với bus APB, chèn 1 chu kỳ chờ, phát yêu cầu đọc/ghi đã hoàn tất sang khối thanh ghi và trả về dữ liệu đọc cùng phản hồi lỗi.
- **timer_register (u_register):** Khối thanh ghi TCR, TDR0/1, TCMP0/1, TIER, TISR, THCSR; giải mã địa chỉ, ghi theo byte strobe, kiểm tra lỗi cấu hình TCR và tạo ngõ ra ngắt tim_int.
- **counter (u_counter):** Gồm hai khối con: **counter_divider** tạo xung cho phép đếm div_tick và tín hiệu halt_ack; **counter_core** chứa bộ đếm 64-bit và bộ so sánh tạo sự kiện ngắt cmp_match_evt.
- **tdr_shadow:** Thanh ghi 64-bit trong timer_top lưu giá trị TDR do phần mềm lập trình; bộ đếm lấy giá trị này khi timer đang tắt.

Luồng dữ liệu chính: lệnh ghi từ CPU đi qua apb_slave → timer_register; các trường cấu hình (timer_en, div_en, div_val, tcmp, halt_req) được đưa tới counter; ngược lại, giá trị đếm counter_val, sự kiện so sánh int_set và halt_ack được đưa về timer_register để phần mềm đọc. Toàn bộ IP dùng chung sys_clk và sys_rst_n.

3.2. Module timer_top

timer_top khởi tạo (instantiate) và nối dây các khối con, đồng thời chứa hai phần logic nhỏ: cổng AND tạo `reg_wr_en_w = reg_valid_w & reg_write_w` và thanh ghi tdr_shadow cập nhật mỗi khi phần mềm ghi TDR0/TDR1. Giao diện bên ngoài của IP gồm cổng clock/reset, bus APB, tín hiệu dbg_mode và ngõ ra ngắt tim_int (Hình 3.2, Bảng 3.1).



Hình 3.2: Khối timer_top.

Tên cổng	Chiều	Độ rộng	Chức năng
sys_clk	input	1 bit	Xung nhịp hệ thống (System clock).
sys_rst_n	input	1 bit	Reset bất đồng bộ toàn IP, tích cực mức thấp.
tim_psel	input	1 bit	APB select.
tim_penable	input	1 bit	APB enable, báo pha ACCESS.
tim_pwrite	input	1 bit	Chiều giao dịch: 1 = ghi, 0 = đọc.
tim_paddr	input	12 bit	Địa chỉ thanh ghi (offset byte).
tim_pwdata	input	32 bit	Dữ liệu ghi.
tim_pstrb	input	4 bit	Byte strobe cho lệnh ghi.
tim_prdata	output	32 bit	Dữ liệu đọc.
tim_pready	output	1 bit	Slave hoàn tất giao dịch (sau 1 chu kỳ chờ).
tim_pslverr	output	1 bit	Báo lỗi lệnh ghi TCR bị từ chối.
dbg_mode	input	1 bit	Hệ thống đang ở chế độ gỡ lỗi.
tim_int	output	1 bit	Ngắt gửi tới CPU (int_st & int_en).

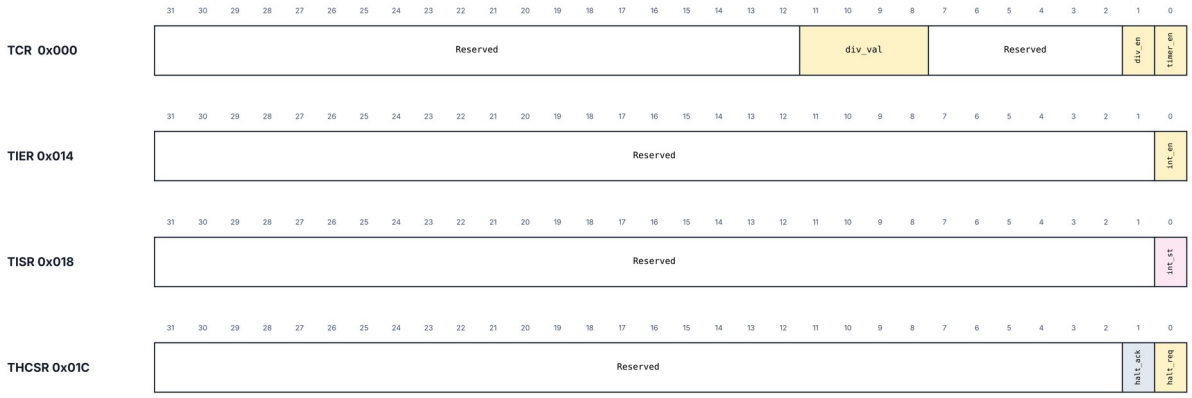
Bảng 3.1: Đặc tả khối timer_top.

3.3. Bản đồ thanh ghi (Register Map)

IP Timer có 8 thanh ghi 32-bit đặt liên tiếp từ offset 0x000 đến 0x01C (Bảng 3.2). Hai bit thấp của địa chỉ được bỏ qua (truy cập theo word). Mọi địa chỉ khác là vùng dự trữ (reserved): đọc trả về 0, ghi bị bỏ qua và không báo lỗi. Cấu trúc trường bit của các thanh ghi điều khiển được thể hiện ở Hình 3.3 và Bảng 3.3.

Offset	Tên	Truy cập	Giá trị reset	Mô tả
0x000	TCR	RW	0x0000_0000	Timer Control Register: timer_en, div_en, div_val.
0x004	TDR0	RW	0x0000_0000	Timer Data Register 0: bộ đếm bit [31:0].
0x008	TDR1	RW	0x0000_0000	Timer Data Register 1: bộ đếm bit [63:32].
0x00C	TCMP0	RW	0xFFFF_FFFF	Timer Compare 0: giá trị so sánh bit [31:0].
0x010	TCMP1	RW	0xFFFF_FFFF	Timer Compare 1: giá trị so sánh bit [63:32].
0x014	TIER	RW	0x0000_0000	Timer Interrupt Enable: bit int_en.
0x018	TISR	RW1C	0x0000_0000	Timer Interrupt Status: bit int_st.
0x01C	THCSR	RW/RO	0x0000_0000	Timer Halt Control/Status: halt_req, halt_ack.

Bảng 3.2: Bản đồ thanh ghi (Register Map) của IP Timer.



Hình 3.3: Cấu trúc trường bit của các thanh ghi TCR, TIER, TISR và THCSR.

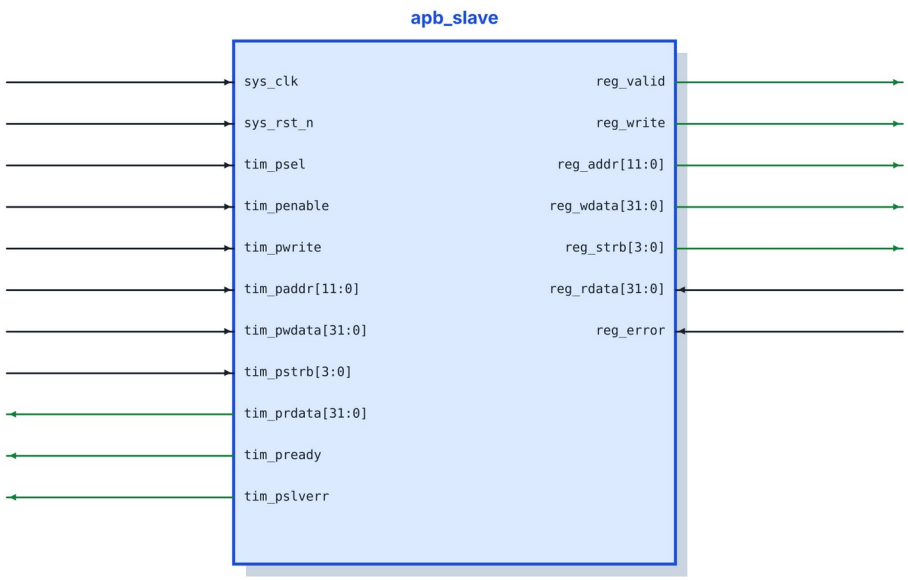
Thanh ghi	Bit	Trường	Truy cập	Reset	Mô tả
TCR	[0]	timer_en	RW	0	Bật bộ đếm. Khi = 0, bộ đếm lấy giá trị TDR đã lập trình.
TCR	[1]	div_en	RW	0	0: đếm mỗi chu kỳ sys_clk; 1: đếm theo bộ chia tần.
TCR	[11:8]	div_val	RW	0	Hệ số chia 2^div_val; 0..7 hợp lệ, 8..15 bị từ chối.
TCR	khác	—	RO	0	Reserved: đọc bằng 0, ghi bị bỏ qua.
TIER	[0]	int_en	RW	0	Cho phép đưa int_st ra ngõ tim_int.
TISR	[0]	int_st	RW1C	0	Cờ ngắt; phần cứng set khi counter = TCMP, ghi 1 để xóa.
THCSR	[0]	halt_req	RW	0	Phần mềm cho phép dừng khi gỡ lỗi.
THCSR	[1]	halt_ack	RO	0	Báo Timer đang dừng: dbg_mode & halt_req.

Bảng 3.3: Mô tả chi tiết các trường bit điều khiển.

3.4. Thiết kế chi tiết module giao tiếp APB (apb_slave)

3.4.1. Module apb_slave và nguyên lý hoạt động

Khối apb_slave (phụ trách: Hoàng Trung Hiếu) là cửa ngõ giữa CPU và IP Timer. Khối xác định pha ACCESS bằng `apb_access = tim_psel && tim_penable` và dùng một flip-flop access_seen để ghi nhớ rằng chu kỳ ACCESS đầu tiên đã trôi qua. tim_pready chỉ lên 1 ở chu kỳ ACCESS thứ hai, tức slave luôn chèn đúng 1 chu kỳ chờ. Yêu cầu sang khối thanh ghi (reg_valid) chỉ được phát ở chu kỳ hoàn tất này, nên mỗi lệnh ghi APB tác động vào thanh ghi đúng một lần (Hình 3.4, Bảng 3.4).



Hình 3.4: Khối apb_slave.

Tên cổng	Chiều	Độ rộng	Chức năng
sys_clk, sys_rst_n	input	1 bit	Clock và reset bất đồng bộ tích cực mức thấp.
tim_psel, tim_penable, tim_pwrite	input	1 bit	Tín hiệu điều khiển giao dịch APB.
tim_paddr	input	12 bit	Địa chỉ APB.
tim_pwdata	input	32 bit	Dữ liệu ghi APB.
tim_pstrb	input	4 bit	Byte strobe APB.
tim_prdata	output	32 bit	Dữ liệu đọc (nối trực tiếp từ reg_rdata).
tim_pready	output	1 bit	apb_access & access_seen.
tim_pslverr	output	1 bit	tim_pready & tim_pwrite & reg_error.
reg_valid, reg_write	output	1 bit	Yêu cầu đã hoàn tất (= tim_pready) và chiều ghi.
reg_addr / reg_wdata / reg_strb	output	12 / 32 / 4 bit	Địa chỉ, dữ liệu, byte strobe sang khối thanh ghi.
reg_rdata	input	32 bit	Dữ liệu đọc từ khối thanh ghi.
reg_error	input	1 bit	Lỗi ghi do khối thanh ghi phát hiện.

Bảng 3.4: Đặc tả khối apb_slave.

3.4.2. Cơ chế chèn chu kỳ chờ và phản hồi lỗi

Flip-flop access_seen bị xóa về 0 ngoài pha ACCESS và lên 1 sau chu kỳ ACCESS đầu tiên. Vì vậy với mọi giao dịch, tim_pready chỉ bằng 1 ở chu kỳ ACCESS thứ hai. tim_pslverr được tạo tổ hợp từ tim_pready, tim_pwrite và tín hiệu lỗi reg_error của khối thanh ghi, nên chỉ có thể bằng 1 ở chu kỳ hoàn tất của một lệnh ghi (Hình 3.5).

```
assign apb_access = tim_psel && tim_penable;
always @(posedge sys_clk or negedge sys_rst_n) begin
    if (!sys_rst_n)       access_seen <= 1'b0;
    else if (!apb_access) access_seen <= 1'b0;
    else                  access_seen <= 1'b1;
end
assign tim_pready  = apb_access && access_seen;
assign tim_pslverr = tim_pready && tim_pwrite && reg_error;
assign tim_prdata  = reg_rdata;
assign reg_valid   = tim_pready;
```

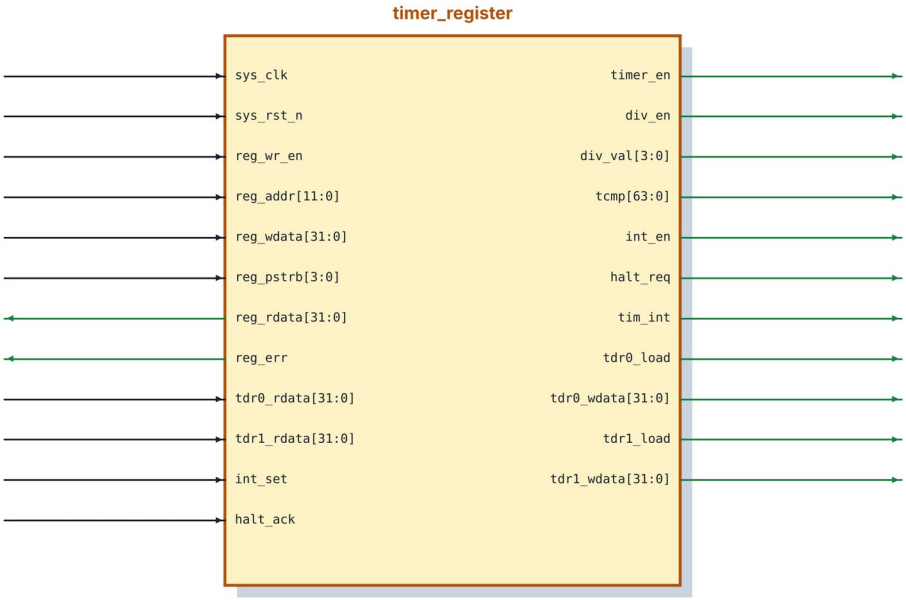
Hình 3.5: Đoạn mã tạo chu kỳ chờ và các tín hiệu phản hồi của apb_slave.

3.5. Thiết kế chi tiết module thanh ghi (timer_register)

3.5.1. Module timer_register và nguyên lý hoạt động

Khối timer_register (tệp register.v, phụ trách: Trần Định Quốc) hiện thực bản đồ thanh ghi ở Bảng 3.2. Địa chỉ được căn theo word (`addr_aligned = {reg_addr[11:2], 2'b00}`) rồi giải mã thành các tín hiệu chọn sel_tcr … sel_thcsr. Dữ liệu đọc được chọn bằng một khối case tổ hợp; TDR0/TDR1 lấy trực tiếp giá trị bộ

đếm hiện tại và bit halt_ack lấy từ khối counter. Lệnh ghi TDR0/TDR1 được chuyển thành xung tdr0_load/tdr1_load kèm dữ liệu đã gộp theo byte để timer_top cập nhật tdr_shadow (Hình 3.6, Bảng 3.5).



Hình 3.6: Khối timer_register.

Tên cổng	Chiều	Độ rộng	Chức năng
sys_clk, sys_rst_n	input	1 bit	Clock và reset bất đồng bộ tích cực mức thấp.
reg_wr_en	input	1 bit	Lệnh ghi APB đã hoàn tất.
reg_addr	input	12 bit	Địa chỉ thanh ghi.
reg_wdata	input	32 bit	Dữ liệu ghi.
reg_pstrb	input	4 bit	Byte strobe.
reg_rdata	output	32 bit	Dữ liệu đọc theo địa chỉ (tổ hợp).
reg_err	output	1 bit	Lỗi ghi TCR (div_val 8..15 hoặc thay đổi khi đang chạy).
timer_en, div_en	output	1 bit	Trường TCR.timer_en, TCR.div_en.
div_val	output	4 bit	Trường TCR.div_val.
tcmp	output	64 bit	Giá trị so sánh {TCMP1, TCMP0}.
int_en, halt_req	output	1 bit	TIER.int_en, THCSR.halt_req.
tim_int	output	1 bit	Ngõ ra ngắt: r_int_st & r_int_en.
tdr0_load, tdr1_load	output	1 bit	Xung ghi TDR0 / TDR1.
tdr0_wdata, tdr1_wdata	output	32 bit	Dữ liệu TDR đã gộp theo byte strobe.
tdr0_rdata, tdr1_rdata	input	32 bit	Giá trị bộ đếm hiện tại (đọc TDR).
int_set	input	1 bit	Sự kiện so sánh từ khối counter.
halt_ack	input	1 bit	Trạng thái dừng gỡ lỗi từ khối counter.

Bảng 3.5: Đặc tả khối timer_register.

3.5.2. Ghi theo byte strobe

Hàm `merge_bytes(old, new, strobe)` tạo giá trị mới bằng cách lấy từng byte của dữ liệu ghi nếu bit strobe tương ứng bằng 1, ngược lại giữ byte cũ. Hàm này được dùng cho TCMP0/TCMP1, TCR, TIER, THCSR và cho dữ liệu ghi TDR (gộp với giá trị bộ đếm hiện tại). Lệnh ghi TISR chỉ có tác dụng khi pstrb[0] = 1.

3.5.3. Kiểm tra lỗi và bảo vệ cấu hình TCR

Giá trị TCR sau khi gộp byte (tcr_merged) được kiểm tra trước khi cập nhật. Lệnh ghi bị coi là lỗi trong hai trường hợp: ghi byte 1 với div_val mới thuộc 8..15 (bit div_val[3] = 1), hoặc timer đang chạy (r_timer_en = 1) mà giá trị mới của div_en hay div_val khác giá trị hiện tại. Khi có lỗi, **toàn bộ lệnh ghi TCR bị bỏ qua** và tim_pslverr = 1 ở chu kỳ hoàn tất. Bảng 3.6 tổng hợp các trường hợp.

Trạng thái	Lệnh ghi TCR	tim_pslverr	Kết quả
timer_en = 0	div_val mới = 0..7	0	Cập nhật timer_en, div_en, div_val.
Bất kỳ	Ghi byte 1 với div_val = 8..15	1	Bỏ qua, TCR giữ nguyên.
timer_en = 1	Đổi div_en hoặc div_val	1	Bỏ qua, TCR giữ nguyên.
timer_en = 1	Giữ nguyên div_en, div_val; đổi timer_en	0	Cập nhật timer_en (dừng timer).
Bất kỳ	Địa chỉ reserved	0	Không thay đổi, đọc lại bằng 0.

Bảng 3.6: Các trường hợp ghi thanh ghi TCR và phản hồi của IP.

3.5.4. Thanh ghi trạng thái ngắt TISR (RW1C)

Cờ r_int_st được xóa khi phần mềm ghi 1 vào bit 0 của TISR với pstrb[0] = 1, và được set khi tín hiệu int_set (sự kiện so sánh từ counter) bằng 1. Trong khối always, câu lệnh set đứng sau câu lệnh xóa nên **sự kiện phần cứng được ưu tiên** khi hai việc xảy ra cùng chu kỳ – thiết kế giữ nguyên thứ tự này từ mã RTL gốc của đề bài (Hình 3.7).

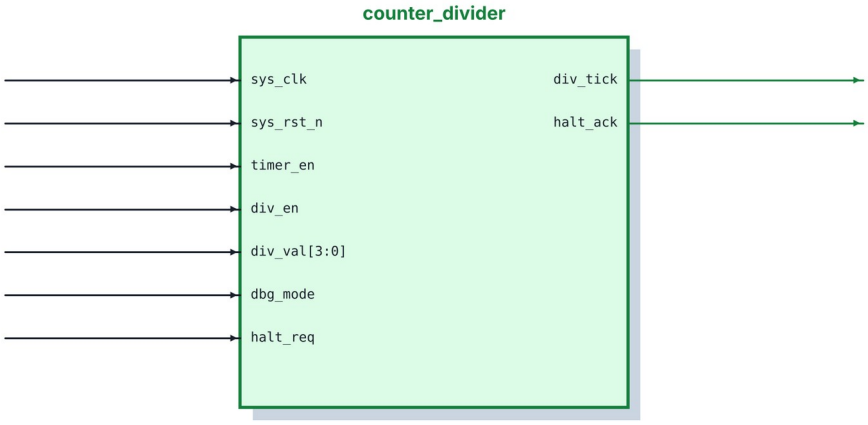
```
assign err_prohibited_divider   = wr_tcr && reg_pstrb[1] && next_div_val[3];
assign err_change_while_enabled = wr_tcr && r_timer_en &&
    ((next_div_en != r_div_en) || (next_div_val != r_div_val));
assign reg_err = err_prohibited_divider || err_change_while_enabled;
...
if (wr_tcr && !reg_err) begin
    r_timer_en <= next_timer_en;
    r_div_en   <= next_div_en;
    r_div_val  <= next_div_val;
end
if (wr_tisr && reg_pstrb[0] && reg_wdata[0]) r_int_st <= 1'b0;  // W1C
if (int_set)                                 r_int_st <= 1'b1;  // HW set
...
assign tim_int = r_int_st && r_int_en;
```

Hình 3.7: Đoạn mã kiểm tra lỗi TCR và cập nhật TISR.

3.6. Thiết kế chi tiết module chia tần (counter_divider)

3.6.1. Module counter_divider và nguyên lý hoạt động

Khối counter_divider (tệp counter.v, phụ trách: Phùng Văn Tân) tạo xung cho phép đếm div_tick cho counter_core và tín hiệu halt_ack. Ngưỡng đếm được tính bằng `div_limit = (8'd1 << div_val) - 8'd1`; bộ đếm 8-bit div_cnt đếm từ 0 tới div_limit rồi quay về 0 và được xóa khi timer_en = 0. Khi `is_halted = dbg_mode && halt_req` bằng 1, div_cnt giữ nguyên giá trị. Ngõ ra `div_tick = timer_en && !is_halted && (!div_en || div_match)`: khi div_en = 0 bộ đếm tăng ở mọi chu kỳ, khi div_en = 1 chỉ tăng ở chu kỳ div_cnt chạm ngưỡng (Hình 3.8, Bảng 3.7).



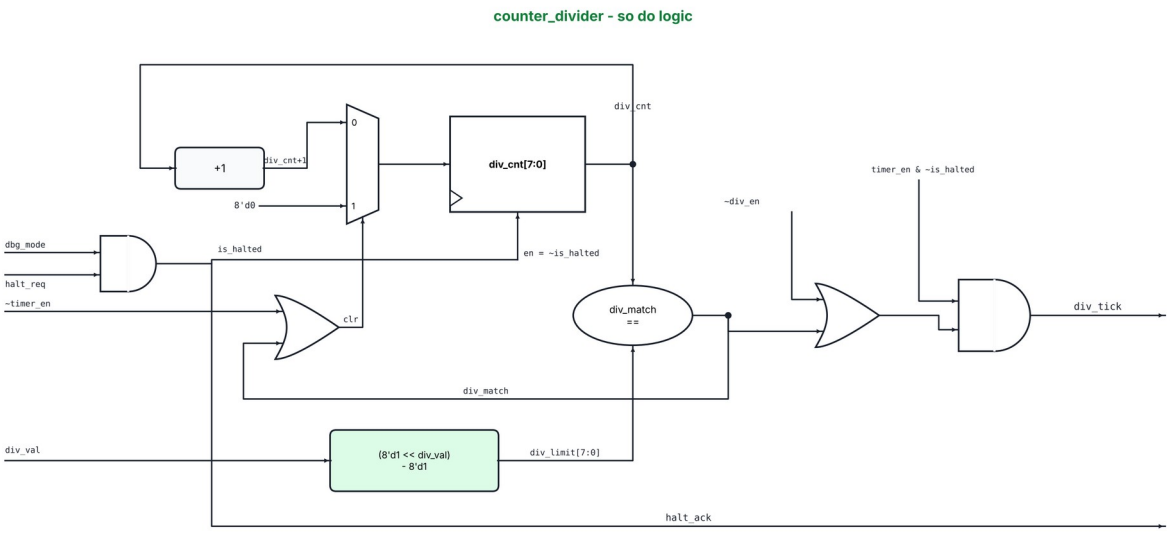
Hình 3.8: Khối counter_divider.

Tên cổng	Chiều	Độ rộng	Chức năng
sys_clk	input	1 bit	Xung nhịp hệ thống.
sys_rst_n	input	1 bit	Reset bất đồng bộ, tích cực mức thấp.
timer_en	input	1 bit	Cho phép timer; = 0 thì xóa div_cnt.
div_en	input	1 bit	Cho phép chia tần.
div_val	input	4 bit	Hệ số chia: tick sau mỗi 2^div_val chu kỳ.
dbg_mode	input	1 bit	Hệ thống đang ở chế độ gỡ lỗi.
halt_req	input	1 bit	Yêu cầu dừng từ THCSR.
halt_ack	output	1 bit	dbg_mode & halt_req.
div_tick	output	1 bit	Xung cho phép đếm rộng 1 chu kỳ (không tạo clock mới).

Bảng 3.7: Đặc tả khối counter_divider.

3.6.2. Sơ đồ logic của bộ chia tần

Sơ đồ logic gồm: khối dịch tạo div_limit, bộ so sánh div_cnt == div_limit, thanh ghi đếm div_cnt với bộ cộng 1 và tín hiệu xóa, cổng AND tạo is_halted và logic tạo div_tick (Hình 3.9). Quan hệ giữa div_val và chu kỳ div_tick được liệt kê trong Bảng 3.8.



Hình 3.9: Sơ đồ logic của khối counter_divider.

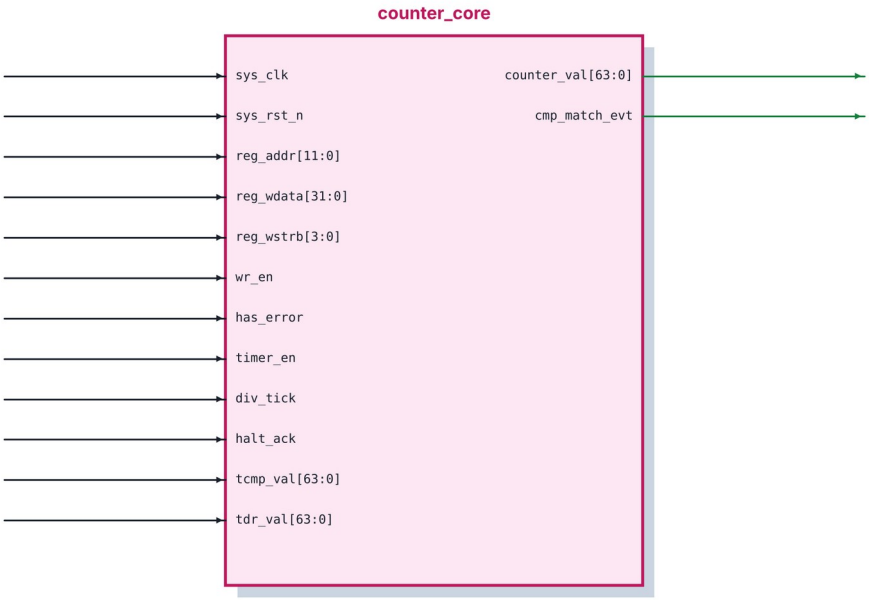
div_val	div_limit	Hệ số chia	Chu kỳ tick (100 MHz)
0	0	1	10 ns
1	1	2	20 ns
2	3	4	40 ns
3	7	8	80 ns
4	15	16	160 ns
5	31	32	320 ns
6	63	64	640 ns
7	127	128	1,28 µs
8 – 15	—	Cấm	Ghi bị từ chối (pslverr = 1)

Bảng 3.8: Quan hệ giữa div_val, ngưỡng đếm và chu kỳ tick (sys_clk = 100 MHz).

3.7. Thiết kế chi tiết module lõi đếm (counter_core)

3.7.1. Module counter_core và nguyên lý hoạt động

Khối counter_core (tệp counter.v, phụ trách: Lê Dương Hoàng Khang) chứa bộ đếm 64-bit counter_val và bộ so sánh. Khối counter bao ngoài khởi tạo hai instance u_counter_divider và u_counter_core, nối chúng qua dây div_tick_wire (Hình 3.10, Bảng 3.9).



Hình 3.10: Khối counter_core.

Tên cổng	Chiều	Độ rộng	Chức năng
sys_clk, sys_rst_n	input	1 bit	Clock và reset bất đồng bộ tích cực mức thấp.
reg_addr	input	12 bit	Địa chỉ ghi (0x004: TDR0, 0x008: TDR1).
reg_wdata / reg_wstrb	input	32 / 4 bit	Dữ liệu và byte strobe ghi TDR.
wr_en, has_error	input	1 bit	Lệnh ghi hoàn tất, lỗi ghi.
timer_en	input	1 bit	= 0: bộ đếm lấy giá trị tdr_val.
div_tick	input	1 bit	Xung cho phép đếm từ counter_divider.
halt_ack	input	1 bit	Dừng đếm khi gỡ lỗi.
tcmp_val	input	64 bit	Giá trị so sánh.
tdr_val	input	64 bit	Giá trị TDR đã lập trình (tdr_shadow).
counter_val	output	64 bit	Giá trị bộ đếm (đọc qua TDR0/TDR1).
cmp_match_evt	output	1 bit	Mức 1 khi counter_val == tcmp_val.

Bảng 3.9: Đặc tả khối counter_core.

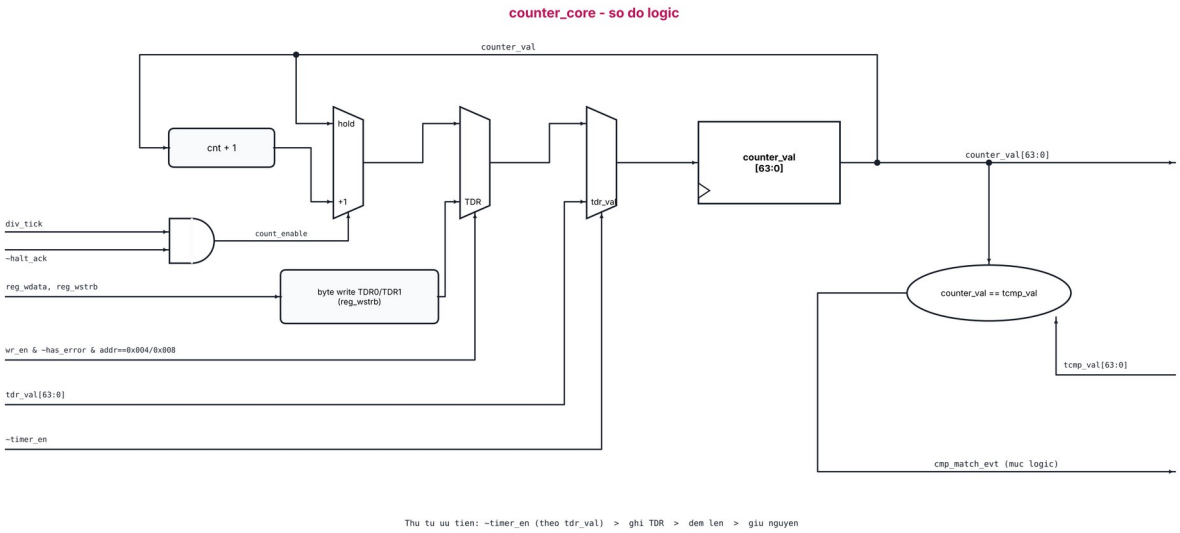
3.7.2. Sơ đồ logic và thứ tự ưu tiên cập nhật bộ đếm

Giá trị bộ đếm được cập nhật tại sườn lên của sys_clk theo thứ tự ưu tiên giảm dần (Hình 3.11, Hình 3.12):

- **Timer tắt (timer_en = 0):** bộ đếm lấy giá trị tdr_val – giá trị TDR mà phần mềm đã lập trình, nhờ đó phần mềm có thể đặt giá trị bắt đầu trước khi bật timer.
- **Ghi TDR khi timer đang chạy:** lệnh ghi hợp lệ vào địa chỉ 0x004/0x008 thay các byte được chọn bởi reg_wstrb của nửa thấp/nửa cao.

- **Đếm lên:** khi `count_enable = div_tick && !halt_ack`, bộ đếm tăng 1 với phép cộng 64-bit đầy đủ.

Bộ so sánh tạo cmp_match_evt = (counter_val == tcmp_val) dưới dạng mức logic; tín hiệu này được đưa về timer_register làm int_set để bật cờ int_st.



Hình 3.11: Sơ đồ logic của khối counter_core.

```
assign cmp_match_evt = (counter_val == tcmp_val);
assign count_enable  = div_tick && !halt_ack;
always @(posedge sys_clk or negedge sys_rst_n) begin
    if (!sys_rst_n)     counter_val <= 64'd0;
    else if (!timer_en) counter_val <= tdr_val;      // theo TDR da lap trinh
    else if (wr_en && !has_error && (reg_addr == 12'h004)) begin
        if (reg_wstrb[0]) counter_val[7:0] <= reg_wdata[7:0];   // ... byte 1..3
    end else if (wr_en && !has_error && (reg_addr == 12'h008)) begin
        if (reg_wstrb[0]) counter_val[39:32] <= reg_wdata[7:0]; // ... byte 1..3
    end else if (count_enable)
        counter_val <= counter_val + 64'd1;
end
```

Hình 3.12: Đoạn mã cập nhật bộ đếm 64-bit của counter_core.

3.8. Tích hợp hệ thống và kết quả tổng hợp logic

Module timer_top (phụ trách tích hợp: Hoàng Quốc Trí) nối các khối con bằng các dây có hậu tố _w (reg_addr_w, timer_en_w, tcmp_w, int_set_w, halt_ack_w…). Ngoài ra, testbench quan sát trực tiếp các tín hiệu nội bộ dut.u_counter.counter_val, dut.u_counter.u_counter_divider.div_tick và dut.u_counter.halt_ack bằng tham chiếu phân cấp để kiểm tra bộ đếm, bộ chia tần và chế độ dừng.

Để khẳng định mã RTL tổng hợp được thành phần cứng và không sinh chốt ngoài ý muốn, thiết kế được tổng hợp thử bằng Yosys 0.33 (thư viện cell chung). Kết quả ở Bảng 3.10 cho thấy toàn bộ IP dùng 210 flip-flop và không có latch nào.

Khối	Flip-flop	Số cell	Ghi chú
apb_slave	1	5	access_seen
timer_register	73	504	TCMP 64 bit, TCR 6 bit, int_en, int_st, halt_req
counter_divider	8	83	div_cnt 8 bit
counter_core	64	552	counter_val 64 bit
timer_top	64	65	tdr_shadow 64 bit
Tổng (timer_top)	**210**	**1.209**	**0 latch**

Bảng 3.10: Kết quả tổng hợp logic (Yosys) của IP Timer.

Chương 4. KẾT QUẢ KIỂM THỬ VÀ MÔ PHỎNG HỆ THỐNG

4.1. Môi trường kiểm thử tự động (Self-checking Testbench)

Để khẳng định IP Timer hoạt động đúng mà không phải dò từng đường sóng thủ công, nhóm xây dựng testbench tự kiểm tra timer_top_tb (tệp timer_top_tb.v) theo kiến trúc ở Hình 2.3. Môi trường hoạt động như một "trọng tài", tự động cấp kích thích và chấm điểm đúng/sai:

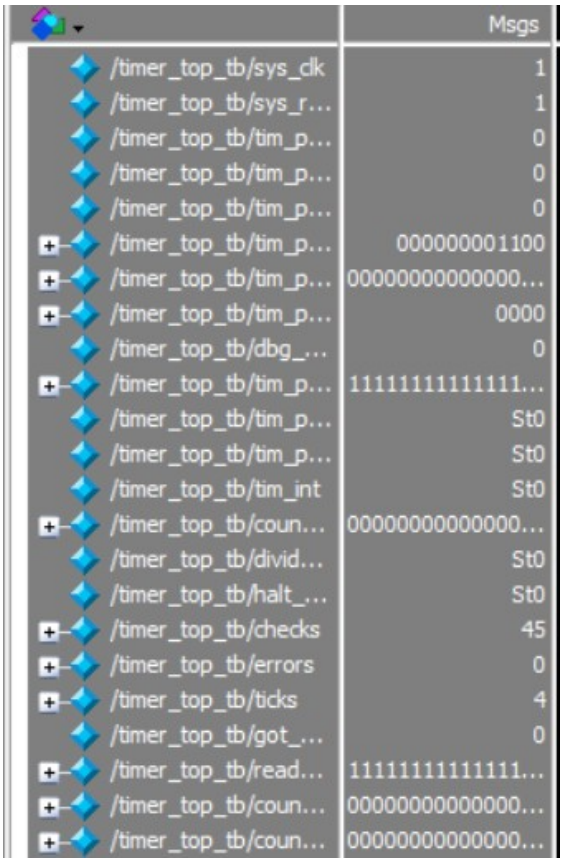
- **Kích thích DUT (Stimulus):** Testbench tạo sys_clk 100 MHz (chu kỳ 10 ns), giữ reset trong 3 chu kỳ đầu rồi đóng vai CPU thông qua hai task apb_write (có byte strobe, trả về pslverr) và apb_read (trả về dữ liệu đọc và pslverr). Các tín hiệu APB được thay đổi ở sườn xuống của clock để tránh tranh chấp với DUT lấy mẫu ở sườn lên.
- **Quan sát nội bộ:** Ba tín hiệu counter_value, divider_tick và halt_ack được lấy trực tiếp từ bên trong DUT bằng tham chiếu phân cấp, dùng để kiểm tra bộ đếm, bộ chia tần và chế độ dừng.
- **Tự động chấm điểm:** Các task check1, check32, check64 so sánh giá trị thực tế với giá trị kỳ vọng; mỗi lần so sánh tăng biến checks, mỗi lần sai tăng biến errors và in [FAIL] kèm thời điểm. Cuối mô phỏng, testbench in dòng TEST SUMMARY với tổng số điểm kiểm tra và số lỗi.
- **Bảo vệ treo mô phỏng:** Một khối watchdog kết thúc mô phỏng sau 100 µs nếu một task APB chờ PREADY vô hạn.

Các nhóm kiểm thử được thiết kế để phủ các chức năng chính của IP (Bảng 4.1).

Nhóm	Nội dung kiểm tra	Số điểm
TEST 1	Giá trị reset của 8 thanh ghi, tim_int = 0 sau reset	9
TEST 2	Ghi theo byte strobe trên TCMP0, địa chỉ reserved (ghi bỏ qua, đọc 0, không lỗi)	6
TEST 3	Cấu hình chia tần hợp lệ, từ chối div_val = 8, khóa div_val khi timer chạy	9
TEST 4	Ghi/đọc TDR0, TDR1 và giá trị bộ đếm 64-bit	3
TEST 5	Sự kiện so sánh, TISR, tim_int, xóa W1C (sai byte lane / đúng byte lane)	5
TEST 6	Debug halt: halt_req đơn lẻ, halt_ack, bộ đếm đóng băng và chạy tiếp	6
TEST 7	Bộ chia tần div_val = 2: đúng 4 xung tick trong 16 chu kỳ	2
TEST 8	Reset bất đồng bộ khi IP đang hoạt động, thanh ghi về giá trị reset	5
	Tổng cộng	**45**

Bảng 4.1: Các nhóm kiểm thử của testbench timer_top_tb.

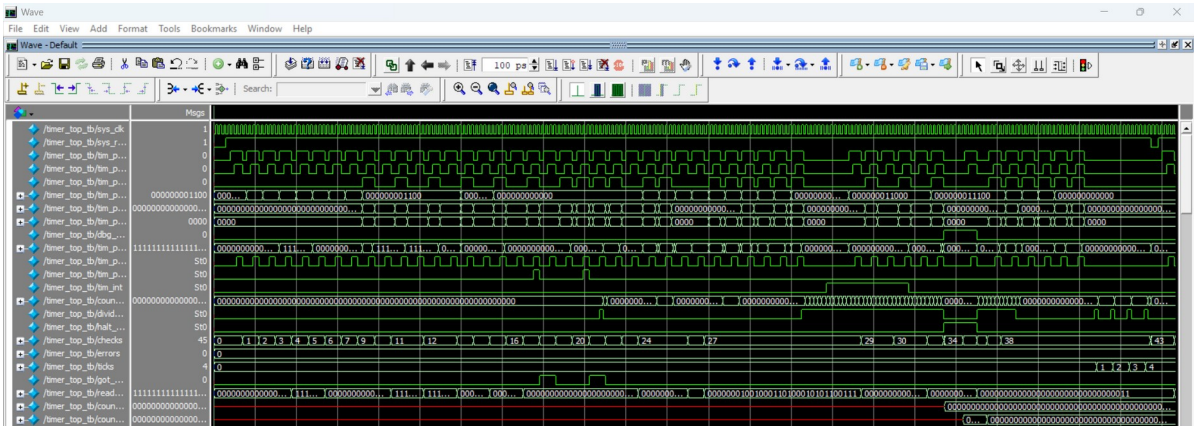
Testbench được biên dịch và mô phỏng trên ModelSim. Cột Msgs của cửa sổ Wave hiển thị giá trị cuối cùng của các biến trong testbench (Hình 4.1): **checks = 45, errors = 0, ticks = 4**, tức toàn bộ 45 điểm kiểm tra đều đạt và bộ chia tần tạo đúng 4 xung tick trong TEST 7.



Hình 4.1: Kết quả kiểm tra tự động của khối timer_top trên ModelSim (checks = 45, errors = 0).

4.2. Kết quả mô phỏng trên phần mềm ModelSim

Hình 4.2 là dạng sóng của các tín hiệu trong testbench timer_top_tb trên ModelSim, trong khoảng thời gian 0 – 2,33 µs bao trùm cả 8 nhóm kiểm thử. Các tín hiệu được hiển thị theo thứ tự khai báo: clock/reset, bus APB (tim_psel, tim_penable, tim_pwrite, tim_paddr, tim_pwdata, tim_pstrb, tim_prdata, tim_pready, tim_pslverr), dbg_mode, tim_int, các tín hiệu quan sát nội bộ (counter_value, divider_tick, halt_ack) và các biến của testbench (checks, errors, ticks, got_error, read_data, count_before, count_after).



Hình 4.2: Mô phỏng Waveform toàn bộ quá trình kiểm thử của timer_top_tb trên ModelSim.

Thời điểm diễn ra từng nhóm kiểm thử trên trục thời gian của Hình 4.2 và các dấu hiệu tương ứng trên dạng sóng được tóm tắt trong Bảng 4.2.

Nhóm	Thời gian	Dấu hiệu trên waveform ModelSim
TEST 1	30 – 350 ns	sys_rst_n lên 1; chuỗi lệnh đọc APB, tim_pready lên 1 ở chu kỳ ACCESS thứ hai.
TEST 2	350 – 670 ns	Lệnh ghi/đọc TCMP0 với tim_pstrb 0001, 0011, 1111; truy cập địa chỉ 0x020.
TEST 3	670 – 1030 ns	Hai xung tim_pslverr (khoảng 775 ns và 895 ns) cho hai lệnh ghi TCR bị từ chối.
TEST 4	1030 – 1190 ns	counter_value nhận giá trị 0x0123_4567_89AB_CDEF từ TDR1:TDR0.
TEST 5	1190 – 1730 ns	counter_value tăng từ 0; tim_int bằng 1 trong khoảng 1485 – 1685 ns rồi về 0 sau lệnh W1C.
TEST 6	1730 – 1910 ns	dbg_mode và halt_ack bằng 1 trong khoảng 1770 – 1850 ns, counter_value đứng yên.
TEST 7	1910 – 2270 ns	divider_tick xuất hiện 4 lần, biến ticks tăng 1 → 4 (2140 – 2260 ns).
TEST 8	2270 – 2370 ns	sys_rst_n xuống 0 trong khoảng 2272 – 2290 ns; các thanh ghi đọc lại về giá trị reset.

Bảng 4.2: Đối chiếu các nhóm kiểm thử với dạng sóng ModelSim.

4.2.1. Kịch bản truy cập APB và thanh ghi (TEST 1 – TEST 4)

Ở phần đầu Hình 4.2, sau khi sys_rst_n lên 1, testbench thực hiện liên tiếp các giao dịch APB. Mỗi giao dịch có dạng SETUP (tim_psel = 1, tim_penable = 0) kéo dài 1 chu kỳ, sau đó là 2 chu kỳ ACCESS; tim_pready chỉ lên 1 ở chu kỳ ACCESS thứ hai – đúng cơ chế chèn 1 chu kỳ chờ của apb_slave. Biến checks tăng dần từ 0 sau mỗi lần so sánh, trong khi errors luôn giữ 0. Trên hàng tim_pslverr có đúng hai xung tương ứng hai lệnh ghi TCR không hợp lệ của TEST 3: ghi div_val = 8 và thay đổi div_val khi timer đang chạy; mọi giao dịch khác đều không báo lỗi.

4.2.2. Kịch bản ngắt so sánh và dừng gỡ lỗi (TEST 5 – TEST 6)

Trong TEST 5, testbench nạp TDR = 0, TCMP = 5, bật int_en rồi bật timer. Khi counter_value đạt giá trị so sánh, cờ int_st được set và tim_int lên 1 (khoảng 1485 ns). tim_int giữ mức 1 trong lúc testbench thử xóa TISR bằng lệnh ghi sai byte lane (pstrb = 0010, cờ vẫn giữ) và chỉ về 0 sau lệnh ghi 1 với pstrb = 0001 (khoảng 1685 ns). Ở TEST 6, khi halt_req đã được đặt và dbg_mode lên 1, halt_ack lên 1 và counter_value đứng yên; khi dbg_mode về 0, halt_ack về 0 và bộ đếm tiếp tục tăng.

4.2.3. Kịch bản bộ chia tần và reset bất đồng bộ (TEST 7 – TEST 8)

Ở TEST 7, timer được cấu hình div_en = 1, div_val = 2. Trên Hình 4.2, divider_tick xuất hiện 4 lần cách đều nhau 4 chu kỳ clock và biến ticks tăng từ 1 lên 4; testbench xác nhận counter_value = 4 sau 16 chu kỳ. Cuối cùng, TEST 8 kéo sys_rst_n xuống 0 khi IP đang hoạt động: tim_pready, tim_int và halt_ack về 0 ngay lập tức; sau khi nhả reset, các lệnh đọc TCR và TCMP0 trả về giá trị reset 0x0000_0000 và 0xFFFF_FFFF. Hai hàng count_before và count_after là biến của testbench, chỉ được gán giá trị từ TEST 6 nên trước đó hiển thị X (màu đỏ) – đây là trạng thái bình thường của biến chưa khởi tạo, không phải lỗi của DUT.

Chương 5. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN CỦA ĐỀ TÀI

5.1. Kết luận

Nhìn lại toàn bộ quá trình thực hiện, nhóm đã xây dựng thành công một lõi IP Timer 64-bit giao tiếp AMBA APB ở mức RTL. Cụ thể, đề tài đạt được các kết quả sau:

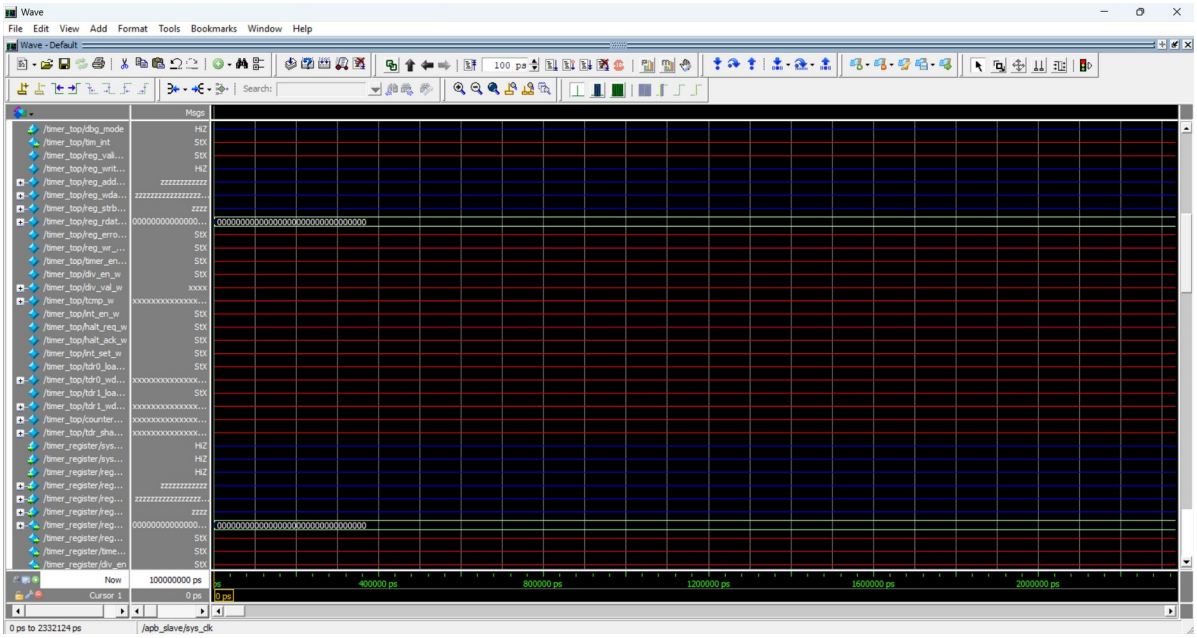
- **Hoàn thiện IP Timer:** Đầy đủ 8 thanh ghi, ghi theo byte strobe, chu kỳ chờ, phản hồi lỗi tim_pslverr, bộ chia tần 2^div_val có bảo vệ cấu hình, bộ đếm 64-bit, ngắt so sánh với cờ RW1C và chế độ Debug Halt.
- **Thiết kế module hóa:** Các khối apb_slave, timer_register, counter_divider, counter_core được tích hợp trong timer_top, chạy trên một miền clock, reset bất đồng bộ, tổng hợp được với 210 flip-flop và không sinh latch.
- **Kiểm chứng trên ModelSim:** Testbench tự kiểm tra với 8 nhóm kiểm thử, 45 điểm kiểm tra cho kết quả checks = 45, errors = 0 trên ModelSim; dạng sóng thể hiện đúng giao thức APB, ngắt, dừng gỡ lỗi, bộ chia tần và reset.
- **Sẵn sàng tái sử dụng:** Mã nguồn, script run.do, project ModelSim và tài liệu được tổ chức thành repository với cấu trúc SRC/ – REPORT/ rõ ràng.

5.2. Những khó khăn trong quá trình thiết kế

Tài liệu tham chiếu chưa thống nhất: Mã RTL mẫu, mô hình GOLDEN_MODEL.py và các tài liệu đặc tả của đề bài mô tả khác nhau ở một số điểm: chu kỳ chia tần (2^div_val hay div_val + 1), giá trị reset của TCR, thứ tự ưu tiên khi xóa và set TISR cùng chu kỳ, hành vi bộ đếm khi timer tắt. Nhóm thống nhất bám theo mã RTL mẫu và GOLDEN_MODEL.py (chia theo 2^div_val, TCR reset bằng 0, sự kiện phần cứng được ưu tiên, bộ đếm theo giá trị TDR khi timer tắt) và ghi rõ các lựa chọn này để trao đổi lại với giảng viên.

Ghi TCR theo byte strobe: Nếu kiểm tra lỗi trực tiếp trên tim_pwdata, một lệnh ghi chỉ chọn byte 0 (bật/tắt timer) có thể bị hiểu nhầm là thay đổi div_val. Vì vậy dữ liệu ghi phải được gộp byte (merge_bytes) với giá trị TCR hiện tại trước khi kiểm tra lỗi; TEST 3 kiểm tra riêng trường hợp bật/tắt timer bằng lệnh ghi chỉ chọn byte 0.

Hiển thị sai dạng sóng trên ModelSim: Ở lần mô phỏng đầu tiên, các module timer_top, timer_register, counter và apb_slave được nạp thêm như những top-level riêng bên cạnh testbench. Hình 5.1 là ảnh chụp ModelSim của lần chạy đó: các tín hiệu của những module này không được nối với nguồn kích thích nào nên hiển thị HiZ (màu xanh dương) và StX (màu đỏ) suốt thời gian mô phỏng.



Hình 5.1: Ảnh chụp ModelSim khi nạp các module RTL như top-level riêng – tín hiệu hiển thị HiZ/StX.

Nhóm khắc phục bằng script run.do chỉ nạp duy nhất testbench timer_top_tb làm top-level (`vsim -voptargs=+acc work.timer_top_tb`) và quan sát tín hiệu nội bộ theo đường dẫn phân cấp /timer_top_tb/dut/…; khi đó dạng sóng hiển thị đúng như Hình 4.2.

5.3. Hướng phát triển trong tương lai

- **Thống nhất với đặc tả:** Cân nhắc chuyển sang quy tắc “xóa của phần mềm được ưu tiên” và tạo sự kiện ngắt theo sườn lên của tín hiệu so sánh để cờ TISR có thể xóa được ngay cả khi bộ đếm dừng đúng tại giá trị TCMP.
- **Đọc 64-bit nhất quán:** Bổ sung cơ chế chụp (snapshot) TDR1 khi đọc TDR0 để phần mềm đọc được giá trị 64-bit nhất quán khi bộ đếm đang chạy.
- **Mở rộng chức năng Timer:** Thêm nhiều kênh so sánh, chế độ tự nạp lại (auto-reload) để tạo ngắt tuần hoàn và chế độ một lần (one-shot).
- **Nâng cấp phương pháp kiểm thử:** Bổ sung mô hình tham chiếu cho toàn bộ IP, kích thích ngẫu nhiên có ràng buộc, đo độ phủ chức năng và SystemVerilog Assertions cho giao thức APB.
- **Đưa xuống phần cứng thực tế:** Tích hợp IP Timer với một lõi CPU RISC-V qua cầu nối AHB/AXI – APB và nạp lên board FPGA.

TÀI LIỆU THAM KHẢO

[1] ARM Limited, “AMBA APB Protocol Specification, Version 2.0”, ARM IHI 0024C, 2010.

[2] IEEE Computer Society, “IEEE Standard for Verilog Hardware Description Language”, IEEE Std 1364-2005, 2006.

[3] Tài liệu đặc tả thiết kế IP Timer (Design Specification, Logic Diagram) và mô hình tham chiếu GOLDEN_MODEL.py, học phần FAP201, Trường Đại học FPT.

[4] Siemens EDA, “ModelSim User’s Manual”.

[5] Nhóm Trương Vũ (lớp IC2007), “Thiết kế và Mô phỏng giao thức truyền thông UART-FIFO”, báo cáo đồ án FAP201 và repository TONG_HOP_CODE_UART (tham khảo cách trình bày).